Estado	Finalizado
Comenzado	miércoles, 7 de octubre de 2026, 17:41
Completado	miércoles, 7 de octubre de 2026, 18:21
Duración	40 mins 17 segs

Pregunta 1

Finalizado

Sin calificar

Explicá qué es el modelo cliente-servidor. Describí el rol de cada actor y quién inicia la comunicación.

Es la arquitectura sobre la que se apoya toda la Web: **dos actores con responsabilidades distintas que dialogan**. Uno pide y el otro responde, siguiendo las reglas del protocolo HTTP.

El cliente siempre inicia la comunicacion. La comunicación empieza con una petición (request) del cliente, el servidor nunca envía datos si el cliente no los pidió.

El cliente es un navegador web (Chrome, Firefox), una app móvil o una aplicación de escritorio. El servidor es una computadora (o grupo de computadoras) que **escucha activamente peticiones** en la red, en puertos estándar: 80 para HTTP y 443 para HTTPS.

Además, siempre que hay una petición hay una respuesta (response).

Es la arquitectura sobre la que se apoya la Web, basada en una división clara de responsabilidades entre dos actores. El cliente (navegador, app móvil o aplicación de escritorio) es el punto de contacto con el usuario. Construye la petición, la envía por la red y muestra (renderiza) la respuesta. El servidor es una computadora, o un grupo de ellas, que escucha peticiones en la red. Procesa la solicitud, aplica la lógica de negocio, consulta bases de datos si hace falta y genera una respuesta estructurada. Además centraliza la información, garantizando seguridad, validación de datos y persistencia. En el HTTP tradicional, la comunicación la inicia siempre el cliente: el servidor nunca envía datos sin que se los hayan pedido.

Pregunta 2

Finalizado

Sin calificar

Un usuario escribe `https://mi-tienda.com/productos` en el navegador. Describí en orden las cinco etapas hasta que ve la página.

1. Resolución DNS
La red no entiende nombres alfanuméricos como mi-tienda.com, solo procesa direcciones IP numéricas. El navegador consulta al servidor DNS resolver: "¿A qué IP numérica corresponde el dominio mi-tienda.com?". El DNS retorna la IP pública exacta del servidor de destino, por ejemplo 190.102.45.12.
2. Conexión TCP y TLS
Se construye la "tubería" cifrada de comunicación antes de transmitir ningún dato.
TCP: establece un canal de transmisión ordenado y confiable mediante el handshake de 3 vías.
TLS: cifra la información enviada; es la capa de seguridad que convierte HTTP en HTTPS.
Diálogo de negociación (handshake): Cliente: "¿Listo para hablar de forma segura?" · Servidor: "Sí, aquí están mis claves de cifrado y mi certificado digital" · Cliente: "Conexión segura establecida".
3. HTTP Request (petición)
Con el canal cifrado listo, el cliente compone un mensaje formal estructurado solicitando el recurso específico. Mensaje intercambiado: GET /productos HTTP/1.1 a la IP 190.102.45.12.
4. HTTP Response (respuesta)
El servidor valida permisos, ejecuta la consulta SQL y arma el payload de resultado. Devuelve el código de estado (status code) junto con el contenido procesado: HTTP/1.1 200 OK + el HTML con el catálogo de productos.
5. Renderizado en el cliente
El trabajo de procesamiento se completa en el navegador:
Descifra los paquetes que viajan por el canal TLS.
Interpreta: lee HTML (estructura), CSS (estilos) y JavaScript.
Dibuja: construye el árbol DOM y dibuja los píxeles en pantalla.
Pide más: inicia peticiones extra por cada imagen o script enlazado.
Carga de recursos secundarios: una página compuesta por 1 HTML y 10 imágenes genera al menos 11 peticiones de red independientes, no una sola solicitud.

1. **DNS**: la red entiende IP y no nombres, así que el navegador consulta al DNS a qué IP corresponde mi-tienda.com, y este devuelve por ejemplo 190.102.45.12.
2. **Conexión TCP/TLS:** se establece una conexión confiable y se cifra la comunicación mediante un handshake, en el que el servidor presenta su certificado y se acuerdan las claves.
3. **Petición HTTP:** el cliente envía un mensaje formal, por ejemplo `GET /productos HTTP/1.1`.
4. **Respuesta HTTP:** el servidor valida la petición, consulta la base de datos y responde con un código de estado y el contenido, por ejemplo `200 OK` con el HTML de la lista de productos.
5. **Renderizado:** el navegador descifra la respuesta, interpreta HTML, CSS y JS, y dibuja la interfaz. Por cada imagen, estilo o script que encuentre, hace nuevas peticiones.

Pregunta 3

Finalizado

Sin calificar

Dado el siguiente mensaje, identificá y explicá cada una de sus partes:

```
POST /api/usuarios HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json
Content-Length: 45

{"nombre": "Carlos", "email": "carlos@mail.com"}
```

Es una petición (request): texto en formato plano que envía el navegador para registrar un nuevo usuario.

Método (POST): acción deseada a realizar.

Ruta (/api/usuarios): recurso en el servidor.

Versión (HTTP/1.1): versión del protocolo.

Cabeceras: metadatos (Host, Content-Type). En el ejemplo de la filmina aparecen Host: api.ejemplo.com, Content-Type: application/json y Content-Length: 45.

Línea en blanco: separador obligatorio.

Cuerpo: objeto JSON con datos ({"nombre": "Carlos", "email": "carlos@mail.com"}).

- **Línea de petición** (`POST /api/usuarios HTTP/1.1`). Tiene el método (POST, la acción que se desea realizar), la ruta (`/api/usuarios`, la dirección exacta del recurso en el servidor) y la versión (HTTP/1.1, el protocolo con que se estructura el mensaje).
- **Cabeceras.** Son metadatos en formato Clave: Valor. `Host` indica el servidor destino y `Content-Type` avisa que los datos enviados son JSON. `Content-Length` indica el tamaño del cuerpo.
- **Línea en blanco.** Es obligatoria e indica al servidor que terminaron las cabeceras.
- **Cuerpo o payload.** Son los datos reales que se envían, en este caso los de un usuario a registrar.

Pregunta 4

Finalizado

Sin calificar

Definí qué es el payload y explicá cómo es la respuesta del servidor al mensaje de la pregunta anterior: qué partes tiene y qué significa el código 201.

Es la "carga útil" del mensaje: los datos sustanciales que realmente se quieren transmitir.

Técnicamente, es el cuerpo (body) de una petición o respuesta HTTP, sin contar los metadatos ni las cabeceras de red.

Funciona en ambos sentidos:

En la petición: son los datos que envía el cliente (formularios, JSON).

En la respuesta: es el recurso solicitado o la confirmación de lo procesado.

El payload (o body) son los datos útiles transmitidos en una petición o respuesta, sin contar cabeceras ni metadatos. En la petición son los datos que envía el cliente (un formulario, un JSON). En la respuesta son el recurso solicitado o la confirmación con los datos procesados.

La respuesta tendría esta forma: `HTTP/1.1 201 Created`, cabeceras (como `Content-Type: application/json`), una línea en blanco y un cuerpo como `{"id": 101, "nombre": "Carlos", "estado": "activo"}`. La línea de estado tiene la versión, el código (un número de 3 dígitos que sintetiza el resultado) y el mensaje (una explicación breve y legible). El código 201 significa que la petición tuvo éxito y se creó un nuevo recurso, y es típico tras un POST. En el cuerpo se devuelve el usuario recién creado con su ID generado.

Pregunta 5

Finalizado

Sin calificar

Explicá qué significa que un método HTTP sea **seguro** y que sea **idempotente**. Clasificá GET, POST, PUT, PATCH y DELETE y justificá con ejemplos al menos dos de ellos.

"seguro" no significa "protegido": en la especificación HTTP significa que el método no modifica datos. Los métodos seguros solo leen o consultan, así que ejecutar un GET 100 veces consecutivas no altera en absoluto la base de datos. Los no seguros alteran o modifican la información almacenada en el servidor backend.

La idempotencia es la propiedad según la cual repetir la misma petición varias veces deja al sistema en exactamente el mismo estado final que si se hubiera hecho una sola vez.

Un método es **seguro** si no modifica el estado del servidor, es decir, solo lee o consulta. Es **idempotente** si ejecutar la misma petición una vez o varias veces seguidas produce el mismo efecto final.

Método	¿Seguro?	¿Idempotente?
GET	Sí	Sí
POST	No	No
PUT	No	Sí
PATCH	No	No (según la especificación, aunque a menudo se implementa como idempotente)
DELETE	No	Sí

Ejemplos que justifican la clasificación:

- **GET:** hacerlo 100 veces no altera la base de datos, por eso es seguro e idempotente.
- **DELETE:** `DELETE /usuarios/5` elimina al usuario, y si se ejecuta cinco veces más el estado final sigue siendo el mismo (el usuario 5 ya no existe), por eso es idempotente pero no seguro.
- **POST:** enviar `POST /pagos` tres veces por error podría generar tres cobros distintos, por eso no es idempotente.